Branch Specialization Checkpoint

Learned Branch Routing

Branch Specialization Project

2026-05-22

Checkpoint reason: unconstrained learned routers did not recover oracle modularity.

Question

Previous branch-isolation result:

- separate branch towers alone stayed functionally entangled;
- oracle routing made local depend on branch 0 and induction depend on branch 1.

This checkpoint asks:

Can a learned router discover role-specific branch modularity without hard-coded local-to-branch-0 and induction-to-branch-1 assignment?

Setup

Same local-vs-induction competition task:

- local copy: `[x, SEP, x]`;
- induction continuation: `[y1,...,y16,y1,...,y16]`;
- local weight 0.25, induction weight 1.0;
- five seeds per config, 1200 training steps.

Four variants:

- `branch_sum`: two branches active everywhere.
- `learned_position_router`: branch gate learned per position.
- `learned_token_router`: branch gate learned from token/position state.
- `oracle_route`: local uses branch 0; induction uses branch 1.

Main Result

Config	Local acc.	Ind. acc.	Same top	Routed match	Branch distance
`branch_sum`	1.0000	0.9997	0.80	0.20	0.0951
`learned_position`	0.9999	0.9987	1.00	0.00	0.0665
`learned_token`	1.0000	0.9989	1.00	0.00	0.0006
`oracle_route`	1.0000	0.9988	0.00	1.00	1.0000

Same top branch means local and induction are causally most dependent on the same branch. Routed match means local top branch is 0 and induction top branch is 1.

Gate Metrics

Config	Gate routed match	Gate distance	Local entropy	Ind. entropy
`branch_sum`	0.00	0.0000	0.6931	0.6931
`learned_position`	0.40	0.0200	0.6924	0.6914
`learned_token`	0.00	0.1045	0.1593	0.3340
`oracle_route`	1.00	1.0000	0.0000	0.0000

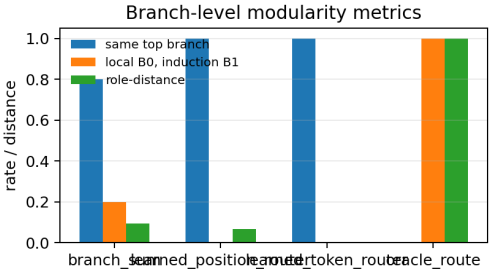
The token router learned lower-entropy gates, but low entropy did not imply role-specific causal modularity.

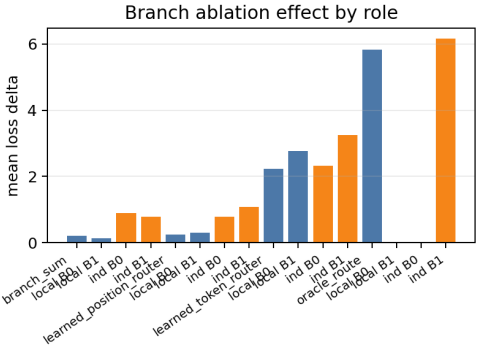
Branch Ablation

Config	Local B0	Local B1	Induction B0	Induction B1
`branch_sum`	0.2017	0.1316	0.8995	0.7907
`learned_position`	0.2437	0.2964	0.7832	1.0852
`learned_token`	2.2221	2.7767	2.3307	3.2506
`oracle_route`	5.8198	0.0000	0.0000	6.1639

Only oracle routing gives the clean local-on-B0 and induction-on-B1 ablation pattern. Learned routers keep the two roles causally co-located or entangled.

Evidence Plot





The learned-router bars sit near the entangled branch-sum regime on causal modularity, while oracle routing remains the positive upper bound.

Interpretation

Supported in this toy setup:

- Learned routers can solve the task.
- Gate specialization and causal modularity can dissociate.
- Explicit routing can produce branch-level functional modularity.

Not supported by this run:

Unconstrained learned routing naturally discovers the role-specific oracle split between local and induction.

Decision

Continue Phase 3, but sharpen the question:

> *What routing signal, regularizer, or task pressure is sufficient for learned functional modularity to emerge, if any?*

Next decisive tests:

- weak router supervision only on scored positions;
- entropy or load-balancing pressure;
- a more conflict-heavy task where shared-branch solutions are less attractive.

Sources and Provenance

- Report: `doc/phase3_toy_learned_router.md`
- Model script: `scripts/toy_branch_isolation_intervention.py`
- Analysis script: `scripts/analyze_branch_isolation.py`
- Run output: `results/phase3_toy_learned_router_lw025/`
- Analysis output: `results/phase3_toy_learned_router_analysis/`
- Deck data snapshot: `presentations/2026-05-22-1249-learned-router/data/`